

← Go Back

Hardware

Nano Every

Tutorials

Nano Every Community Projects

Connecting Two Nano Every Boards Through I2C

Communicating with Four Boards Through UART with Nano Every

Connecting Two Nano Every Boards Through UART

Home / Hardware / Nano Every / Communicating with Four Boards Through UART with Nano Every

Communicating with Four Boards Through UART with Nano Every

Learn how to set up one Nano Every board to communicate four other boards using UART.

Author · Fabricio Troya · Last revision · 07/17/2024

ON THIS PAGE

Communicating with Four Boards Through UART

- Goals
- Hardware & Software Needed
- UART Communication
- SoftwareSerial Library +
- Creating the Program +
- Testing It Out +
- Conclusion

Communicating with Four Boards Through UART

In this tutorial we will control the built-in LED on four Arduino Nano Every boards, from a main Arduino Nano Every. To do so, we will connect the boards using a wired communication protocol called UART.

Note: This example would work by connecting an Arduino Nano Every board with any other Arduino board, but be mindful that both board must work at the same voltage. If the operating voltage differs between the connected boards, the board with the lower operating voltage could be damaged.

In this example, we will power the Arduino boards through the computer, then we will use the Serial Monitor to send some commands to the main Nano Every board, that will be connected through the UART with others Nano Every boards. Depending on the commands received by the main Nano Every board, it will turn **ON** or **OFF** the built-in LED on different boards.

Goals

The goals of this project are:

- Understand what the UART is.

- Use the SoftwareSerial library

- Use UART communication between five Arduino boards.

Hardware & Software Needed

For this project we will need:

- Arduino Nano Every**

- 4 x Arduino Nano Every (or any other Arduino boards that works at 5V)

- 5 x **mini breadboard**

- 13 x **jumper wires**

UART Communication

the most used device-to-device communication protocols. It allows an asynchronous serial communication in which the data format and transmission speed are configurable. The UART communication sends data bits one by one, from the least significant to the most significant, framed by start and stop bits so that precise timing is handled by the communication channel.

Embedded systems, microcontrollers, and computers mostly use UART as a form of device-to-device hardware protocol. Among the available communication protocols, UART uses only two wires for its transmitting and receiving ends, TX (Transmitter) and RX (Receiver). These pins are dedicated for that specific purpose, either transmitting or receiving.

Asynchronous means there is no clock signal to synchronize the output bits from the transmitting device going to the receiving end, but the baud rate needs to be the same on both the transmitting and receiving devices. The baud rate, is the rate at which information is transferred to a communication channel. In the serial port context, the set baud rate will serve as the maximum number of bits per second to be transferred.

UART is the communication protocol we use to communicate from the computer to the board

used for communication with the computer, which means connecting anything to these pins can interfere with that communication, including causing failed uploads to the board. This is not the case for the Nano or MKR families, since these boards have two separate channels, using **Serial** for the communication with the computer and **Serial1** for the communication with any other device through UART.

If you want to learn more about the UART protocol and how it works, you can check [this link](#).

SoftwareSerial Library

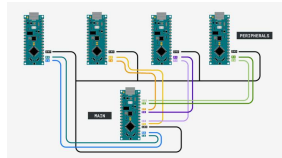
The Arduino hardware has built-in support for UART communication on pins 0 and 1. The `SoftwareSerial` library has been developed to allow serial communication on other digital pins of the Arduino, using software to replicate the functionality (hence the name "SoftwareSerial").

It is possible to have multiple software serial ports with a speed up to 115200 bps, but in the Nano Every board the maximum speed is limited to 9600 bps.

Circuit

In order to communicate all the Arduino boards, we will need to

Note: If you want to initialize the UART communication with any other Arduino board, please check [here](#) the serial port and the pins you need to use.



Connecting four peripherals to a main board.

At the moment of making the connections, we need to remember that this protocol has two dedicated lines TX and RX, so we need to make sure to connect the TX pin of the main board, with the RX of the other one. The same goes for the RX pin, which need to be connected to the TX pin of the other board.

To finish, it is very important that we connect the GND pins of all boards to each other. If we don't do this, the voltage reference will be different for each one of the boards so the communication won't work as intended.

Note: In order to enable serial communication, the Arduino boards must be connected to your computer via USB.

Creating the Program

1. Configuring the Peripheral boards

First, let's connect one of the Arduino Nano Every board to the computer and open the Arduino Cloud Editor. This board will act as a peripheral, which means that it will only receive data from the main board and turn ON or OFF the built-in LED according to the received values. To start with the programming, start a new sketch and name it

Nano_4_UART_receiver.

1.1. Receiver code walkthrough

Let's start inside the `setup()` function. We need to initialize the built-in LED as `OUTPUT` and then turn it off. Then, we need to initialize the UART communication with the main board using the `Serial1.begin()` function since the receiver board will receive the data through the **UART1** port.

```
1 void setup() {  
2   pinMode(LED_BUILTIN,  
3   digitalWrite(LED_BUIL  
4  
5   Serial1.begin(9600);  
6 }
```

Note: Remember that if you used a different board to the ones used on this tutorial, you will need to check the pins you need to use, they may vary depending on the board.

Inside the `loop()` we need to

on the buffer using the `Serial1.available()` function inside a `while` statement. This configuration will run the portion of code that is inside the brackets of the `while()`, only if there is any byte available to be read on the buffer of the UART1. If that is the case, we will read and store the available data in the `receivedData` variable using the `Serial1.read()` function, since we want to read the incoming data from the main Arduino board.

```
1 void loop() {  
2   while (Serial1.available()) {  
3     char receivedData =
```

Now, we need to add a `if...else` statement to check if the received data is `'1'` or `'2'` in order to turn on or off the built-in LED respectively.

Note: When we use UART communication, all the data transmitted is text formatted, which means that we need to use character logic instead of numeric logic for compatibility. For example, in order to know if the data received is a "5" we need to check with the ASCII character of 5 ('5') instead of the numeric value of 5 (5). Therefore, we use `dataReceived == '5'` instead of `dataReceived == 5`.

```
1  if (receivedData == '1'
2      digitalWrite(LED_
3  )
4      else if (receivedDa
5          digitalWrite(LED_
6      )
7  )
8  }
```

1.2. Uploading the code to the peripherals

Once we have finished the code, let's upload it to the first peripheral Arduino Nano Every board. This board is now programmed to act as a peripheral in this scenario. Once the code is uploaded, let's connect the next peripheral board to the computer.

All the boards that act as a peripheral will use the same sketch, the only thing we need to change is the value that will trigger the built-in LED. In the table below we can see the value use for each board.

Board	Value to turn ON the LED	Value to turn OFF the LED
Peripheral 1	'1'	'2'
Peripheral 2	'3'	'4'
Peripheral 3	'5'	'6'
Peripheral 4	'7'	'8'

not necessary for it to stay connected to the computer in order to function. However, it needs to be powered in order to work.

2. Configuring the Main board

It is time to open a new sketch and name it as

Nano_4_UART_transmitter, and include the **SoftwareSerial** library using the

```
#include <SoftwareSerial.h>
```

function. Now we need to name the new UART ports and set the pins that we will use for each port. In order to make the coding easier we will call the UART ports as `Serial2`, `Serial3` and `Serial4`.

```
1 #include <SoftwareSerial.h>
2
3 SoftwareSerial Serial2(
4 SoftwareSerial Serial3(
5 SoftwareSerial Serial4(
```

Note: This pins can not be selected randomly since each port has assigned specific pins internally. For more information about the pins you can use for each board, please check [here](#)

Inside the `setup()`, we will start initializing the built-in LED as we did in the code for the peripheral boards.

```
1 void setup() {  
2   pinMode(LED_BUILTIN,  
3   digitalWrite(LED_BUIL
```

In addition, we need to initialize the Serial communication with the computer and all the peripheral boards using `Serial.begin()` functions. After that, we will use a `Serial.println()` function to print a message in the Serial Monitor.

```
1 Serial.begin(9600);  
2   Serial1.begin(9600);  
3   Serial2.begin(9600);  
4   Serial3.begin(9600);  
5   Serial4.begin(9600);  
6  
7   Serial.println("Enter  
8 }
```

In the `loop()`, we will use an `if` statement to check if there is any incoming byte to read from the Serial Monitor using the `Serial.available()` function. If it is, it will be stored in the `inByte` variable using the `Serial.read()` function. We then use a `switch case` statement to create an action for each number entered in the Serial Monitor.

```
1 void loop() {  
2   // check if there is  
3   if (Serial.available()  
4     int inByte = Serial  
5  
6   switch (inByte){
```

Each `case` will turn on or off a different built-in led thanks to the `Serial.println()` functions that will send the data through different UART ports. Then, we need to add a `delay()` of 100 milliseconds in order to have enough time to send the data. Once the data are sent, we will use a `Serial.print()` function to print a message in the Serial Monitor about which board has been turned on or off and finally we use `break` statement to end the case and start the next one.

Send to receiver 1

```
1 // Send to receiver 1  
2   case '1':  
3     Serial1.println(  
4       delay(100);  
5     Serial.print("Bo  
6     break;  
7  
8     case '2':  
9     Serial1.println(  
10    delay(100);  
11    Serial.print("Bo  
12    break;
```

Send to receiver 2

```
1 //Send to receiver 2
2     case '3':
3         Serial2.println(
4             delay(100);
5             Serial.print("Bo
6             break;
7
8     case '4':
9         Serial2.println(
10            delay(100);
11            Serial.print("Bo
12            break;
```

Send to receiver 3

```
1 //Send to receiver 3
2     case '5':
3         Serial3.println(
4             delay(100);
5             Serial.print("Bo
6             break;
7
8     case '6':
9         Serial3.println(
10            delay(100);
11            Serial.print("Bo
12            break;
```

Send to receiver 4

```
1 //Send to receiver 4
2     case '7':
3         Serial4.println(
4             delay(100);
5             Serial.print("Bo
6             break;
7
8     case '8':
9         Serial4.println(
10            delay(100);
11            Serial.print("Bo
12            break;
```

```
1 //Send to all boards
2     case '9':
3         Serial1.println(
4             delay(100);
5         Serial2.println(
6             delay(100);
7         Serial3.println(
8             delay(100);
9         Serial.print("Al
10        break;
11
12     case '0':
13         Serial1.println(
14             delay(100);
15         Serial2.println(
16             delay(100);
17         Serial3.println(
18             delay(100);
19         Serial4.println(
20             delay(100);
21         Serial.print("Al
22        break;
```

Lastly, we need to end the `switch case` statement using a `default :` function. This part of the code will be called when the `inByte` doesn't match with any of the cases. So, we will only add a `Serial.println()` function to move the incoming text in the Serial Monitor one line down and finally the `break` statement as in the other cases.

```
1 default:
2     Serial.println("
3     break;
4 }
5 }
6 }
```

Once we have finished the code, let's upload it to our main Arduino Nano Every board. This board is now programmed to act as the main board.

3. Complete code

If you choose to skip the code building section, the complete code for both the receivers and the transmitter can be found below:

Receivers:

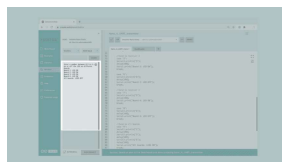
```
1  void setup() {
2    pinMode(LED_BUILTIN,
3    digitalWrite(LED_BUI
4
5    Serial1.begin(9600);
6  }
7  void loop() {
8    while (Serial1.avail
9      char receivedData
10     // Change the valu
11     // Receiver 1: '1'
12     // Receiver 2: '3'
13     // Receiver 3: '5'
14     // Receiver 4: '7'
15     if (receivedData ==
16       digitalWrite(LED
17     }
18     else if (receivedD
19       digitalWrite(LED
20     }
21   }
22 }
```

Transmitter:

```
1  #include <SoftwareSerial.h>
2
3  SoftwareSerial Serial1(2, 3);
4  SoftwareSerial Serial2(4, 5);
5  SoftwareSerial Serial3(6, 7);
6
7  void setup() {
8    pinMode(LED_BUILTIN, OUTPUT);
9    digitalWrite(LED_BUILTIN, LOW);
10
11    Serial1.begin(9600);
12    Serial2.begin(9600);
13    Serial3.begin(9600);
14    Serial4.begin(9600);
15
16    Serial.println("Arduino Serial Test");
17 }
18
19 void loop() {
20   // check if there is data available
21   if (Serial1.available()) {
22     int inByte = Serial1.read();
23
24     switch (inByte) {
25       // Send to Serial2
26       case '1':
27         Serial2.println("1");
28       case '2':
29         Serial2.println("2");
30       case '3':
31         Serial2.println("3");
32       case '4':
33         Serial2.println("4");
34       case '5':
35         Serial2.println("5");
36       case '6':
37         Serial2.println("6");
38       case '7':
39         Serial2.println("7");
40       case '8':
41         Serial2.println("8");
42       case '9':
43         Serial2.println("9");
44       case '0':
45         Serial2.println("0");
46       default:
47         Serial2.println("");
48     }
49   }
50 }
```

Testing It Out

After you have successfully verified and uploaded the sketch to the boards, make sure the main board is connected and then open the Serial Monitor. You need to enter a number between 0-9 to turn ON or OFF a built-in LED in any peripheral board.



Serial Monitor interface
for turning ON/OFF

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon.

- Arduino board connected to the wrong port.

- Connection between the Arduino boards are not correct.

- Missing the initialization of a UART port.

- Accidental interruption of cable connection.

Conclusion

In this simple tutorial we learned how to connect four Arduino boards to a main Arduino Nano Every board so that they can communicate using UART communication.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

